Functional Requirements Document (FRD)

Project Name: Construction Materials & Operations SaaS

Version: 1.0

Target Platform: Mobile (iOS/Android) via React Native

Backend Stack: NestJS (Node.js), PostgreSQL

Architecture: Offline-First (Local Database + Sync)

Primary User Persona: Site Superintendent

---

1. Executive Summary

This project aims to build a mobile-first SaaS platform designed to bridge the gap between field operations (Daily Logs) and the supply chain (Materials Marketplace). The application is built on an **Offline-First** architecture to ensure full functionality in remote construction sites with unstable internet. It empowers Site Superintendents to document site progress, manage attendance, receive materials, and procure supplies directly from the field.

---

2. User Persona: The Site Superintendent

- **Role:** Responsible for daily site operations, safety, and schedule adherence.
- **Environment:** Dusty, noisy, outdoors, often with poor or no cellular data.
- **Pain Points:**
  - Spending hours entering data into Excel after a long shift.
  - Losing data due to connectivity drops.
  - Material delays causing work stoppages.
- **Goals:** Quick data entry (under 15 mins/day), instant material ordering, and bulletproof reliability.

---

3. Technical Architecture

3.1 High-Level Stack

- **Frontend (Mobile):** React Native (Expo/CLI).
- **Local Database (Device): WatermelonDB** (preferred for React Native performance with large datasets like BOQs) or SQLite.
- **Backend API:** NestJS (TypeScript).
- **Central Database:** PostgreSQL.
- **Object Storage:** AWS S3 (for site photos).

3.2 Offline-First Sync Protocol

The app treats the **Local Database** as the single source of truth for the UI.

1. **Write:** User actions (e.g., submitting a log) write instantly to WatermelonDB.
2. **Queue:** If offline, the action is marked as `unsynced`.
3. **Sync:** A background job detects network availability and pushes changes to the NestJS `sync` endpoint.
4. **Conflict Resolution:** "Last Write Wins" (LWW) strategy based on server-side timestamps.

---

4. Functional Requirements

Module A: Advanced Daily Logs

Focus: Speed, Accuracy, and Compliance.

REQ-DL-01: Automated Weather Logging

- **Feature:** Auto-fetch weather data based on GPS location.
- **Flow:**
    1. User opens "Daily Log."

2. App calls OpenWeatherMap API (if online) to fetch Temp, Humidity, Wind Speed.
3. If offline, User manually inputs data or app retries later.
4. **Data Point:** Critical for justifying weather-related delays.

REQ-DL-02: Site Attendance (Labor Tracking)

- **Feature:** Track headcount and hours for own crew and subcontractors.
- **Input:**
  - Select `Subcontractor Company` from dropdown.
  - Input `Headcount` (e.g., 5 Carpenters).
  - Input `Total Hours Worked`.
- **Validation:** Prevent submission if hours exceed logical max (e.g., 24h * headcount).

REQ-DL-03: Progress Documentation (Photos)

- **Feature:** Visual evidence of work completed.
- **Flow:**
  1. User captures photo via in-app camera.
  2. **Meta-tagging:** App automatically tags photo with:
     - GPS Location (Lat/Long).
     - Timestamp.
     - User ID.
  3. **Offline Handling:** Image is stored locally (FS). A background worker uploads to S3 when online and updates the record with the URL.

REQ-DL-04: Material Receipt (GRN - Goods Receipt Note)

- **Feature:** Digital verification of incoming deliveries.
- **Flow:**

1. User selects open `Purchase Order (PO)` from list.
2. User checks off items received (e.g., "Ordered 100 bags, Received 50").
3. User snaps photo of the paper delivery ticket.
4. **System Action:** Updates Inventory Level and creates a "Partial Delivery" record.

Module B: Materials Marketplace

Focus: Procurement and Supply Chain Integration.

REQ-MP-01: Vendor Discovery & Catalog

- **Feature:** Searchable database of verified suppliers.
- **UI:** Grid view of materials (Concrete, Steel, Electrical) with filtering by "Distance to Site" and "Rating."
- **Data:** Syncs a lightweight version of the catalog to local DB for offline browsing.

REQ-MP-02: Request for Quotation (RFQ)

- **Feature:** Convert BOQ shortages into RFQs.
- **Flow:**
  1. User selects material category (e.g., "Cables").
  2. User inputs quantity needed.
  3. User selects "Blast RFQ" to top 3 nearest vendors.
- **Backend:** NestJS creates a `RFQ` record and triggers notifications to Vendor Apps.

REQ-MP-03: Direct Ordering & Tracking

- **Feature:** Place orders for consumables or urgent items.
- **Status Tracking:**
  - `Order Placed` → `Confirmed` → `Out for Delivery` → `Delivered`.

- **Geofencing:** App alerts Superintendent when the delivery truck is within 1km of the site.

---

5. Database Schema (PostgreSQL)

The schema must support the offline sync protocol (using `created_at` and `updated_at` for conflict resolution).

5.1 Table: `daily_logs`

Column	Type	Description
`id`	UUID	Primary Key
`project_id`	UUID	FK to Projects
`user_id`	UUID	Superintendent ID
`log_date`	DATE	Date of activity
`weather_data`	JSONB	{ temp: 30, condition: "Sunny" }
`status`	ENUM	`DRAFT`, `SUBMITTED`, `SYNCED`
`updated_at`	TIMESTAMP	Used for Sync Protocol

5.2 Table: `log_photos`

Column	Type	Description
`id`	UUID	Primary Key
`daily_log_id`	UUID	FK to Daily Log
`local_path`	TEXT	Path on mobile device (temp)
`s3_url`	TEXT	Cloud URL (final)
`gps_lat`	FLOAT	Geo-tagging
`gps_long`	FLOAT	Geo-tagging

5.3 Table: `material_orders`

Column	Type	Description
`id`	UUID	Primary Key

`vendor_id`	UUID	FK to Vendor
`items`	JSONB	Array of items [{item_id, qty, price}]
`total_amount`	DECIMAL	Financial value
`delivery_status`	ENUM	`PENDING`, `SHIPPED`, `DELIVERED`

6. API Definition (NestJS)

6.1 Controller: `SyncController`

- `POST /sync/push`: Receives JSON payload of local changes (created/updated records) from WatermelonDB.
- `GET /sync/pull`: Returns records modified since `last_pulled_at` timestamp.

6.2 Controller: `MarketplaceController`

- `GET /materials`: Returns paginated list of materials (cached locally).
- `POST /orders`: Creates a new order.
- `GET /orders/:id/track`: Returns current GPS location of delivery driver.

7. Non-Functional Requirements

1. **Offline Capability:** The app must open and allow full data entry (Logs, Photos) without *any* network connection.
2. **Sync Reliability:** Data must automatically sync within 60 seconds of network restoration.
3. **Photo Compression:** Photos must be compressed locally before upload to save bandwidth (Target: <500KB per image).
4. **Security:** All offline data must be encrypted (SQLCipher) to prevent access if the device is lost.

8. Roadmap & Phasing

- **Phase 1:** Core Daily Logs (Attendance, Weather, Photos) + Offline Sync.

- **Phase 2:** Material Marketplace (Browsing & RFQs).
- **Phase 3:** Full Ordering Integration & Fintech (BNPL for materials).